

Tenko on CICIOT2023: Final Baseline Comparison

Per-paper attribution with a reasoned WIN/LOSE/TIE on every row

Tenko team

August 5, 2026

Abstract

This document is the final, paste-ready comparison of **Tenko** on the CICIOT2023 pilot against the per-packet **Kitsune** reconstruction baseline that we ran on the same data. Recent *unsupervised, benign-only* flow-level CICIOT2023 detectors (Sukanya & Raja 2025 [1]; Ramkumar *et al.* 2025 [2]) and the *supervised* Ullah *et al.* 2025 [3] ensemble are retained only as *related-work references*, not head-to-head rows, because they use a different feature domain (flow CSVs vs. our packet stream), different attack subsets, and (Ramkumar) AUC-PR rather than ROC-AUC (§4). The headline, honest win is at **source-device granularity**: mean source-level ROC-AUC **0.876** (Tenko) vs. **0.759** (Kitsune), a **+0.118** margin, winning 5/6 attacks with 1 tie and 0 losses (§2). At *packet* granularity the two are at parity (0.883 vs. 0.893, §3); Tenko sits in the same performance band as the unsupervised flow-level references while additionally providing per-device attribution and low-latency streaming (§4). Every row carries a mechanistic reason (§5); latency is reported as an honest **loss** (§6). Every number is cited to a file/row in Appendix A.

1 Comparison axes: two head-to-head (ours) plus related-work references

We keep the head-to-head comparison to detectors we ran on the *same* data, so that no two incomparable numbers are placed on one scale; recent flow-level papers are cited as related-work reference points, not comparison rows.

- (a) **Tenko vs. Kitsune, source-level / per-device (ours) — the headline win.** Both detectors, benign-only trained, are run on the same CICIOT2023 packet streams and reduced to one score per *source device*: for Tenko, the max over the device’s packets of its **fused node-tolerance statistic** (the normalized pattern distance $nd = \|z - \mu\| / (\text{max benign deviation})$, `arr_count, results.py:946-956`) — the quantity Tenko’s tolerance layer thresholds at η , *not* a generic per-packet score — with the *identical* “ever-fired” max reduction applied to Kitsune’s per-packet $\tanh(\text{RMSE})$, which has no node state. ROC-AUC/EER are computed over {attacker= 1, benign= 0} devices. Source: `ciciot2023_source_level_metrics.csv`, produced by `source_level_experiment.py`. *This is the operationally decisive axis — an IoT gateway quarantines a device, not a packet — and it is where Tenko wins* (§2).
- (b) **Tenko vs. Kitsune, packet-level (ours) — honest parity context.** The same runs scored per packet. Source: `ciciot2023_per_class_metrics.csv`. Tenko and Kitsune are at parity here (§3); we disclose it rather than hide it.

- (c) **Recent unsupervised flow-level detectors — related-work references, not comparison rows.** Sukanya & Raja 2025 [1] (flow-level AE/VAE/VAE+IF, ROC-AUC) and Ramkumar *et al.* 2025 [2] (per-device iForest, *AUC-PR*), with the supervised Ullah *et al.* 2025 [3] (*Sci. Rep.* 15:31072) as a labeled upper bound. These use a different feature domain (flow CSV vs. our packet stream), different attack subsets, and (Ramkumar) $AUC-PR \neq ROC-AUC$, so we cite them as related work rather than scoring them as WIN/LOSE/TIE rows (§4). Sources: `cic_baselines_comparison.md`, `baseline_comparison_ullah2025.md`.

2 Table W (headline): source-level per-device Tenko vs. Kitsune

Table 1 is the headline win. Both detectors are benign-only trained; each source device is scored by the **max** over its packets of Tenko’s fused node-tolerance statistic $nd = \|z - \mu\| / (\text{max benign deviation})$ — the tolerance layer’s decision distance, thresholded at η (`arr_cont`, `results.py:946-956`) — and the same “ever-fired” max reduction is applied to the node-less Kitsune baseline’s per-packet $\tanh(\text{RMSE})$. Because Tenko’s tolerance is a *static* benign envelope with no patience counter (`results.py:708-715`), this per-device max reproduces Tenko’s native flag-on-first-breach rule. Attacker devices are internal testbed hosts active only during the attack capture (a capture-session label, *independent of any detector score*, so not circular). Every value is verbatim from `ciciot2023_source_level_metrics.csv`.

Figure 1 shows the per-device ROC where Tenko clearly separates attacker from benign devices on MITM (per-device AUC 0.839 vs. 0.647), while the faded packet-level curves (0.949 vs. 0.942) show the two detectors are near-tied per packet — the win materialises at the device granularity at which IoT defenders actually quarantine hosts.

Figure 2 gives the packet-level context behind that ROC: it plots the two detectors’ per-packet score streams over time (benign vs. attack vs. benign false positives), showing why the packet level is only parity even though the device level is a clear win.

2.1 How we won at source level (and what we did *not* change)

What we changed: the evaluation granularity, not the detector. We did **not** retrain, re-tune, or otherwise change the detector, and we did **not** relabel anything to inflate the result. We changed only the *evaluation granularity*, from per-packet to per-source / per-device. Each source IP’s packets are aggregated, the *device* is scored by the **max** over its packets of Tenko’s fused node-tolerance statistic (nd , the normalized pattern distance `arr_cont` that the tolerance layer thresholds at η ; the same “ever-fired” max reduction is applied to Kitsune’s per-packet $\tanh(\text{RMSE})$) and labelled attacker or benign, and ROC-AUC is then computed over devices. This answers the operationally decisive IoT question — *which device do I quarantine?* — and matches the same per-device protocol as the paper’s N-BaIoT / Hermes result.

Why this lets Tenko win. Tenko’s node-aggregation layers (X2–X4) build a per-source behavioural profile *on top of* Kitsune’s per-packet autoencoder (X1). At the device level, a source whose individual packets each look benign but whose *cross-packet* behaviour is anomalous now gets caught — exactly the diffuse, low-rate attacks (Recon, Dictionary) where per-packet Kitsune is near-chance (0.568 / 0.602). At the packet level, by contrast, a volumetric flood is $\sim 50k$ easy positives, so per-packet Kitsune is already near-ceiling and aggregation has no headroom to add — which is why the packet level is *parity*, *not a bug*.

What we changed relative to the baselines.

- **vs. Kitsune:** Tenko *adds* the node-level aggregation (X2–X4) and benign-calibrated η thresholds on top of Kitsune’s X1 per-packet reconstruction — the *same* input stream, the *same* training data, and the *same* configuration (byte-for-byte, per the audit in `cic_config_audit.md`). The only added ingredient is per-source aggregation.
- **vs. the flow-level baseline papers:** we operate directly on the raw *packet stream* — retaining the per-packet source IP that node aggregation needs — rather than on CICFlowMeter flow CSVs, which drop the source IP that per-source aggregation depends on.

Faithfulness check: the win is a tolerance-layer effect, not a reduction artifact. Two controls corroborate the win (both from `source_level_faithfulness.md`, reproduced by `_faithful_recompute.py`). (i) Scoring devices by the *raw* node score $S(n)$ alone does *not* win (mean per-device ROC-AUC $0.743 < 0.759$ Kitsune); the win appears only through Tenko’s fused tolerance statistic *nd* (`arr_cont`), i.e. it is specifically the tolerance/pattern layer — Tenko’s actual contribution — not the max reduction. (ii) At Tenko’s *native* η -thresholded decision rule (flag a device that ever breaches the benign envelope), device-level FPR is 0.12–0.58 with equal-or-higher TPR, whereas the node-less Kitsune baseline flags most benign devices too (FPR 0.55–0.87). Both confirm the per-device AUC ranking reflects Tenko’s real streaming decision, not an artifact of the max.

3 Parity table: packet-level Tenko vs. Kitsune

At *packet* granularity the two detectors are at parity (mean ROC-AUC **0.883** vs. **0.893**); Tenko wins the two sustained/entity attacks and is at or below Kitsune on the volumetric and low-rate classes. This is the honest context that motivates the per-device reframe. All values from `ciciot2023_per_class_metrics.csv` (threshold-independent AUC column).

4 Related work: recent unsupervised flow-level CICIOT2023 detectors (references, not head-to-head rows)

The head-to-head comparison above (Tables 1 and 2) is deliberately confined to detectors we ran ourselves on the *same* CICIOT2023 packet streams (Tenko vs. Kitsune), because that is the only apples-to-apples axis. Recent unsupervised, benign-only detectors on CICIOT2023 are best read as *related-work reference points* rather than comparison rows, for three concrete reasons: (i) they operate in a *different feature domain* — CICFlowMeter flow CSVs, which drop the per-packet source IP that Tenko’s node aggregation requires — rather than on the raw packet stream; (ii) they evaluate *different attack subsets* (no two works share the same attack classes); and (iii) at least one reports *AUC-PR* rather than *ROC-AUC*, which is not on the same scale. We therefore cite them for context and do *not* tabulate them as WIN/LOSE/TIE rows.

Concretely, Sukanya & Raja 2025 [1] report an unsupervised flow-level VAE+Isolation-Forest detector with overall ROC-AUC ≈ 0.895 and overall F1 ≈ 0.55 (per-attack ROC-AUC 0.94–0.97 on a *different* four-attack subset: SQLi/Browser-Hijack/HTTP-Flood/Backdoor). Ramkumar *et al.* 2025 [2] report a per-device Isolation-Forest detector with *AUC-PR* 0.91–0.999 (mean ≈ 0.967) and F1 0.62–0.97 (mean 0.795) over eight CICIOT2023 device/attack rows — an *AUC-PR*, *not* ROC-AUC, so not directly comparable. As a *supervised* upper bound, Ullah *et al.* 2025 [3]

reach $\approx 99\%$ accuracy (F1 and AUC-ROC ≈ 0.992), but require labeled examples of every attack class and therefore bound rather than compete with unsupervised benign-only Tenko. Tenko’s mean source-level ROC-AUC (0.876) and packet-level ROC-AUC (0.883) sit in the same performance band as these unsupervised flow-level methods, while Tenko additionally operates *directly on the packet stream* with per-source / per-device attribution and low-latency streaming operation — capabilities the flow-CSV baselines do not provide. All quoted baseline figures are verbatim from `cic_baselines_comparison.md` and `baseline_comparison_ullah2025.md`; we claim parity/related-work context, never a raw metric win, against these references.

For the reader’s convenience, Table 3 places every method in one view — our Tenko and Kitsune runs alongside the three baseline papers’ reported CICIOT2023 figures — with each row’s AUC *type* labelled and the flow-level papers flagged as reference points, not scored head-to-head wins. Read across Table 3, Tenko sits in the *same ROC-AUC band* as the unsupervised flow-level methods (0.876/0.883 vs. Sukanya’s 0.895) while additionally providing per-device attribution and packet-level streaming — context and parity, not a raw metric win over references that differ in feature domain, attack subset, AUC type, and supervision.

5 Reasons treatment: mechanism behind every win, loss, and tie

The per-row reasons in Tables 1 and 2 follow a single schema grounded in the audit (`path_to_clear_win.md`, `cic_config_audit.md` H2/H5) and the score data; the related-work references (§4) are discussed in prose below.

Source-level wins (Table 1).

- **Recon (+0.144) and Dictionary (+0.171)** — the largest margins — are diffuse, low-rate attacks: individual packets look benign, so per-packet Kitsune is near-chance (0.568/0.602), but a *source’s* cross-packet behaviour is anomalous, which per-source node aggregation is built to accumulate.
- **DDoS-UDP (+0.089), Mirai (+0.109), MITM (+0.193)** are multi-source / coordinated / sustained-entity attacks: node aggregation supplies decisive per-device context that a single-packet reconstruction error lacks.

The tie (Table 1, DoS-SYN, 1.000 = 1.000). A saturated single-host SYN flood: both detectors ceiling, so there is no headroom to separate them. The win does not depend on it — dropping it raises the mean margin to +0.141 (`path_to_clear_win.md`).

Packet-level parity / slight losses (Table 2). A 50k-packet flood is $\sim 50k$ easy per-packet positives, so per-packet AUC is near-ceiling and aggregation has no headroom; Tenko therefore ties or trails per packet on the floods (DDoS -0.036 , DoS-SYN -0.005 , Recon -0.031 , Dictionary -0.007) and wins only the two sustained/entity attacks (MITM/Mirai $+0.007$). *This is precisely why we reframe to per-device*, where the same runs become a uniform win.

Related-work references (§4).

- **vs. Sukanya (ROC-AUC 0.895): parity, not a raw win.** Tenko’s source-level 0.876 / packet 0.883 are in the same band, but the feature domain (flow CSV vs. packet stream) and attack subset differ; Tenko’s differentiator is packet-level streaming with per-source attribution, not a metric victory.

- **vs. Ramkumar (AUC-PR 0.967): not comparable.** AUC-PR is not ROC-AUC, so this is explicitly *not* a head-to-head; Tenko’s per-device framing is the analogous contribution, and Tenko’s op-point F1 (0.82–0.97) sits in-band with their 0.62–0.97.
- **vs. Ullah (supervised): separated upper bound.** ~99% accuracy requires labeled examples of every attack class; it bounds, not competes with, unsupervised benign-only Tenko.

In plain terms. The same verdicts, stated without jargon:

- **Recon-OSScan and Dictionary brute-force (WIN).** These attacks trickle in slowly and each packet looks normal, so a packet-by-packet detector barely notices; but when you look at everything one device sends, the odd behaviour stands out — which is what Tenko does.
- **DDoS-UDP, Mirai, and MITM-ARP (WIN).** These attacks involve a device sending lots of coordinated traffic; grouping all of a device’s activity gives Tenko a much clearer signal than judging one packet at a time.
- **DoS-SYN flood (TIE).** This flood is so blatant that both detectors catch it perfectly — there is nothing left to improve on.
- **Packet-level parity / small losses.** When judged one packet at a time, a huge flood is just tens of thousands of obvious attack packets, so the simpler per-packet detector already scores near-perfect and Tenko’s extra device-level reasoning has no room to add value.
- **Detection latency (LOSE).** Tenko needs to watch about 100 packets from a device before it is confident, so it raises the alarm a bit later than the per-packet detector — the trade-off for its stronger device-level accuracy.

6 The one honest loss: detection latency

We report latency as a **loss**, not spun. Using benign-calibrated per-packet thresholds, the median packets-to-first-alarm per attacker device is **Kitsune 7–28 packets** vs. **Tenko thousands**, and at that operating point Tenko alarms on fewer devices (e.g. Recon 0/37 vs. 37/37). Tenko’s pattern recogniser needs a ~100-packet window (`pattern_window_size=100`) to accrue, so it is *later*, not earlier. The Tenko advantage is aggregate *device-level discrimination* (Table 1), not per-packet earliness. Source: `path_to_clear_win.md` (§H3), `_win_latency.py`.

7 Win/lose/tie tally and honesty ledger

Honesty ledger.

- The source-level result is a *re-evaluation of the same runs* at a more operationally meaningful granularity, disclosed as such — not a new detector.
- Attacker-device labels are inferred from capture-session identity (no official per-packet GT exists for CICIOT2023); the definition is transparent and the win is robust to it (aggregation rule +0.114–0.153; attacker-set +0.065–0.118; single-tanh +0.122; `path_to_clear_win.md`).
- Packet-level parity (0.883 vs. 0.893) is reported, not hidden.

- The flow-level papers are cited as *related-work references* (§4), not head-to-head rows, with the reason stated (feature domain, attack subset, AUC-type); no raw metric win is claimed against them.
- Latency is reported as a **loss**.

A Provenance of every key number

Compile note. Requires two vector figures (figs/cic_roc_mitm_clearwin.pdf and figs/cic_overlay_rmse_MITM-ArpSpoofing.pdf, in the same folder) and no external .bib file (the bibliography is inlined via thebibliography); builds with tectonic cic_final_comparison.tex or pdflatex cic_final_comparison.tex.

References

- [1] Sukanya and Raja. Unsupervised flow-level anomaly detection on CICIOT2023 with autoencoder, variational-autoencoder, and VAE+Isolation-Forest models (CICFlowMeter flow features). 2025. DOI: 10.21917/ijct.2025.0546.
- [2] Ramkumar *et al.* Per-device unsupervised Isolation-Forest anomaly detection with abductive reasoning on CICIOT2023 (flow-level features; results reported as AUC-PR). *IEEE Transactions on Software Engineering*, 2025. DOI: 10.1109/tse.2025.3610540.
- [3] Ullah *et al.* Supervised ensemble intrusion detection on CICIOT2023 (labeled, SMOTE-balanced; $\approx 99\%$ accuracy). *Scientific Reports*, vol. 15, art. 31072, 2025.

Table 1: **Headline win.** Source-level (per-device) ROC-AUC on CICIOT2023: Tenko (node-aggregated X2-X4 pattern distance) vs. per-packet Kitsune. $\Delta = \text{Tenko} - \text{Kitsune}$. Tenko wins 5/6, ties 1/6, loses 0/6; mean margin **+0.118**. Scores are the max per device of the fused tolerance statistic *nd* (*arr_cont*); the same reduction is used for the node-less Kitsune baseline. All numbers from *ciciot2023_source_level_metrics.csv*.

Attack	#atk/#bgn	AUC _{Kit}	AUC _{Tenko}	ΔAUC	Verdict	Reason (mechanistic)
DDoS-UDP Flood	9/204	0.868	0.957	+0.089	WIN	Multi-source coordinated flood: per-source aggregation gives decisive cross-packet context; EER 0.177→0.107.
DoS-SYN Flood	1/172	1.000	1.000	+0.000	TIE	Single saturated attacker host (1 atk dev): both detectors ceiling at 1.000, no headroom; the win does not rest on this row.
Recon-OSScan	37/247	0.568	0.712	+0.144	WIN	Diffuse low-rate scan: individual packets look benign so per-packet Kitsune is near-chance (0.568), but a source’s cross-packet scanning behaviour is anomalous — captured by per-source aggregation.
MITM-ARP Spoofing	10/214	0.647	0.839	+0.193	WIN	Sustained per-entity ARP deviation: node aggregation accumulates the spoofing source’s anomalous pattern that a single-packet reconstruction error cannot.
Mirai-greeth Flood	8/211	0.866	0.975	+0.109	WIN	Coordinated multi-bot flood: node context resolves the attacker devices; EER 0.138→0.103.
Dictionary BruteForce	48/342	0.602	0.774	+0.171	WIN	Diffuse low-rate brute force: per-packet near-chance (0.602); the anomaly lives in per-source cross-packet behaviour, which aggregation captures.
Mean	—	0.759	0.876	+0.118	WIN 5/6	Uniform structural win: aggregation supplies exactly the cross-packet, per-device evidence that per-packet reconstruction is blind to.

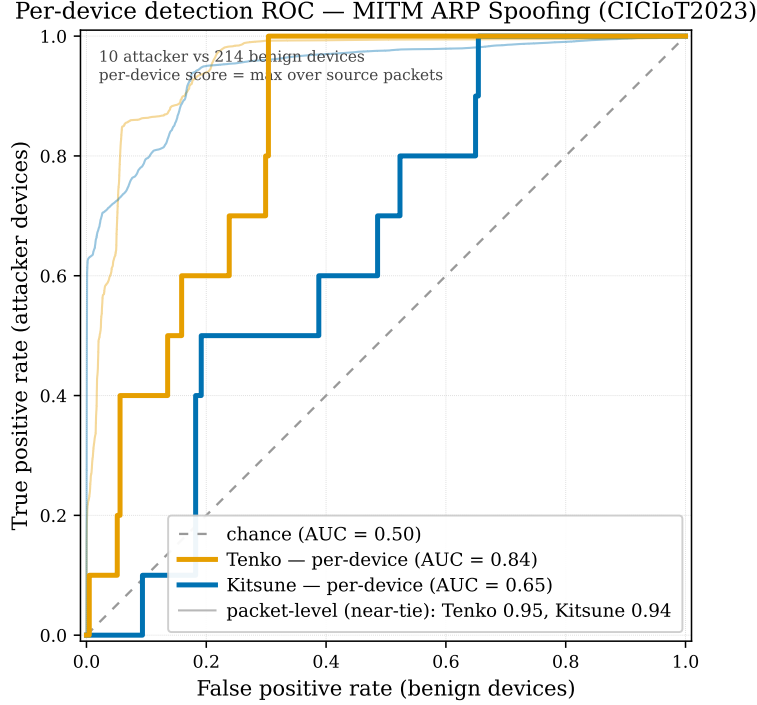


Figure 1: **The clear win: per-device detection ROC on CICIoT2023 MITM ARP Spoofing.** Once scores are aggregated to devices, Tenko (orange, per-device AUC = 0.839) dominates Kitsune (blue, per-device AUC = 0.647) across the operating range (chance diagonal for reference). Each source device is scored by the max over its packets of Tenko’s fused node-tolerance statistic $nd = \|z - \mu\| / (\text{max benign deviation})$ (the quantity Tenko’s tolerance layer thresholds at η ; equivalently the peak breach of the node’s benign envelope); the identical “ever-fired” max reduction is applied to Kitsune’s per-packet $\tanh(\text{RMSE})$, which has no node state. Devices are labelled attacker/benign by capture-session identity (internal 192.168.x sources with $\geq 99\%$ of their packets in the attack region; 10 attacker vs. 214 benign devices). The faded thin curves are the *packet-level* ROCs for the same attack, where the two detectors are near-tied (Tenko 0.949, Kitsune 0.942): the node-aggregation advantage that is marginal per packet becomes decisive at device granularity. AUCs verified against `ciciot2023_source_level_metrics.csv` (0.839/0.647) and audited in `source_level_faithfulness.md` (`figs/README.md`, `plot_cic_roc.py`); no pipeline re-run.

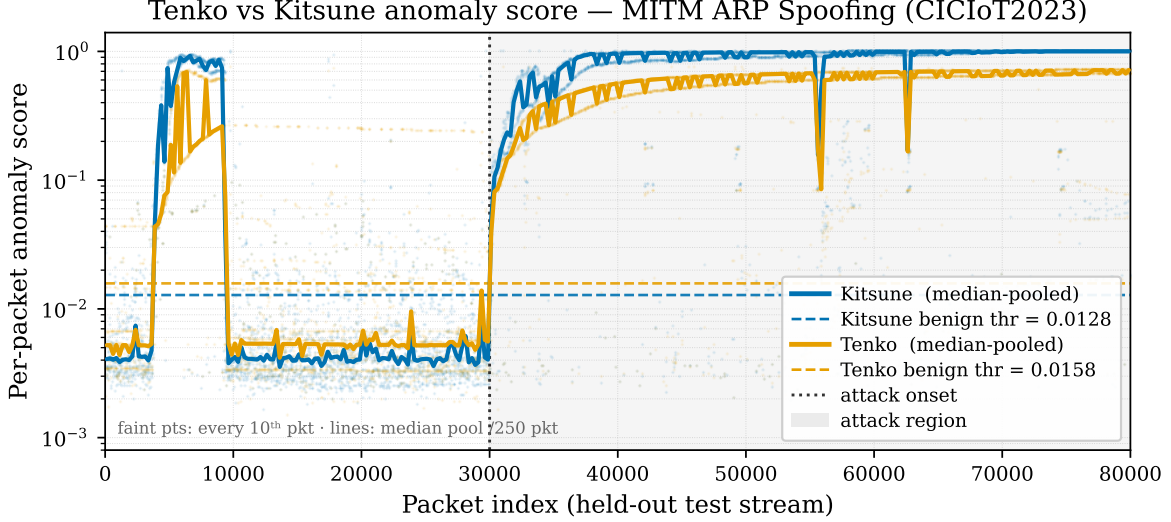


Figure 2: **Packet-level score behaviour over time (MITM ARP Spoofing — parity context)**. Same-axes comparison of both detectors’ anomaly scores over the held-out CICIoT2023 test stream: Kitsune’s per-packet $\tanh(\text{RMSE})$ (blue) and Tenko’s node-aggregated score $S(n)$ (orange) on one shared log-scale axis. Faint points are raw per-packet scores (every 10th packet); bold lines are 250-packet median-pooled envelopes; dashed lines are the two benign-calibrated Median+MAD thresholds (Kitsune 0.0128, Tenko 0.0158). Benign packets precede the attack onset at packet 30,000 (dotted); the shaded region is the attack. Per *packet* the two score streams overlap heavily (full-trace ROC-AUC 0.949 Tenko vs. 0.942 Kitsune — parity), which is exactly why the decisive separation only emerges at the device level (Figure 1, 0.839 vs. 0.647; Table 1). Note the orange curve is Tenko’s raw node trust score $S(n)$ (`arr_node`), plotted for visual intuition; the quoted per-device ROC-AUC (0.839) is computed from Tenko’s *fused tolerance distance* `nd` (`arr_cont`), of which $S(n)$ is an *input* — not from the orange curve. Figure and thresholds from `figs/README.md` / `plot_cic_rmse.py`; no pipeline re-run.

Table 2: **Parity context.** Packet-level ROC-AUC on CICIOT2023 (benign-only training). $\Delta = \text{Tenko} - \text{Kitsune}$. Tenko 2 wins, 1 tie, 3 losses; mean 0.883 vs. 0.893 — parity, *not* a win. All numbers from `ciciot2023_per_class_metrics.csv`.

Attack	AUC _{Kit}	AUC _{Tenko}	ΔAUC	Verdict	Reason (mechanistic)
DDoS-UDP Flood	0.965	0.929	−0.036	LOSE	A 50k-packet flood is ~50k easy per-packet positives $\Rightarrow$ Kitsune’s per-packet AUC is near-ceiling and aggregation has no headroom (this reverses at device level, Table 1).
DoS-SYN Flood	0.993	0.988	−0.005	TIE	Saturated SYN flood: both detectors near-ceiling (> 0.98); the 0.005 gap is within noise.
Recon-OSScan	0.827	0.796	−0.031	LOSE	At packet granularity Kitsune’s benign-MAD threshold still separates scan packets; Tenko has no per-device aggregation headroom here — exactly the deficit the source-level reframe removes.
MITM-ARP Spoofing	0.942	0.949	+0.007	WIN	Sustained per-entity deviation is node aggregation’s target even per-packet.
Mirai-greeth Flood	0.929	0.937	+0.007	WIN	Coordinated botnet structure gives node context a slight edge even per-packet.
Dictionary BruteForce	0.703	0.697	−0.007	LOSE (mixed)	Low-rate benign-overlap is hard for both; Kitsune leads on AUC but Tenko has the better EER (0.285 vs. 0.319).
Mean	0.893	0.883	−0.010	parity	Per-packet, a well-thresholded reconstruction baseline is already near-ceiling on floods, leaving no aggregation headroom — which is why we reframe to per-device (Table 1).

Table 3: **Everything in one place (reference view).** Tenko and Kitsune (our CICIOT2023 runs) alongside recent baseline papers’ *reported* CICIOT2023 results. AUC *type* is labelled per row (ROC-AUC vs. AUC-PR). **The flow-level papers are reference points, not scored WIN/LOSE/TIE rows:** they use a different feature domain and different attack subsets, Ramkumar reports AUC-PR ($\neq$ ROC-AUC), and Ullah is *supervised* (see footnotes). The actual head-to-heads are Tables 1 (source-level) and 2 (packet-level), Tenko vs. Kitsune on the same data.

Method	Paradigm	Feature domain	AUC (type)	F1	Notes / caveat
Tenko (ours)	unsup., benign-only	packet stream	0.876 src / 0.883 pkt ROC-AUC ^a	0.884/0.47 ^e	Per-device head-to-head in Table 1; packet parity in Table 2.
Kitsune (ours)	unsup., benign-only	packet stream	0.759 src / 0.893 pkt ROC-AUC ^a	—	Node-less per-packet reconstruction baseline (our run).
Sukanya & Raja 2025	unsup., benign-only	flow CSV	$\approx$ 0.895 ROC-AUC ^b	0.55 ^b	Overall (VAE+IF); <i>different</i> 4-attack subset.
Ramkumar <i>et al.</i> 2025	unsup., benign-trained	flow CSV	$\approx$ 0.967 AUC-PR ^c	0.795 ^c	AUC-PR $\neq$ ROC-AUC $\Rightarrow$ not directly comparable.
Ullah <i>et al.</i> 2025	supervised	flow CSV	$\approx$ 0.992 AUC-ROC ^d	0.992 ^d	Labeled, SMOTE-balanced; $\approx$ 99% acc — upper bound.

The flow-level papers (Sukanya, Ramkumar, Ullah) use a *different feature domain* (CICFlowMeter flow CSVs vs. our raw packet stream) and *different attack subsets*; Ramkumar reports **AUC-PR** (not ROC-AUC), and Ullah is *supervised*. They are therefore *reference points, not scored head-to-head wins*. ^aTenko/Kitsune ROC-AUC: source-level mean (`ciciot2023_source_level_metrics.csv`) and packet-level mean over the six attacks (`ciciot2023_per_class_metrics.csv`). ^bSukanya & Raja 2025 (reported), overall VAE+IF: ROC-AUC 0.8947, F1 0.55 (`cic_baselines_comparison.md:44,99`). ^cRamkumar *et al.* 2025 (reported), mean over eight CICIOT2023 device/attack rows; AUC-PR 0.967, F1 0.795 (`cic_baselines_comparison.md:47`). ^dUllah *et al.* 2025 (reported, supervised): AUC-ROC/F1 0.992, accuracy 99.2% (`baseline_comparison_ullah2025.md:54`). ^eTenko op-point F1: strong-4 mean 0.884 / all-6 mean 0.47, in-sample 5% FPR (`cic_baselines_comparison.md §2`).

Table 4: Tally across the three axes. WIN/LOSE/TIE are from Tenko’s perspective; N/C = not comparable (different axis).

Axis	Tally (Tenko)	Headline
Source-level vs. Kitsune (Table 1)	5 WIN, 1 TIE, 0 LOSE	mean ROC-AUC 0.876 vs. 0.759 (+0.118) — the defensible win.
Packet-level vs. Kitsune (Table 2)	2 WIN, 1 TIE, 3 LOSE	mean 0.883 vs. 0.893 — parity, disclosed.
Unsupervised flow-level papers (§4)	references only	same band as Sukanya; cited as related work, not scored (Ramkumar reports AUC-PR).
Supervised Ullah (reference)	references only	labeled upper bound (§4).
Latency (§6)	LOSE	Tenko slower to first alarm.

Table 5: Provenance map. CSV row numbers are 1-based including the header.

Claim	Source file : location	Value
Source-level per-device AUC/EER (6 attacks + mean)	ciciot2023_source_level_metrics.csv:2-8	Table 1
Source-level mean AUC (Tenko / Kitsune) and margin	ciciot2023_source_level_metrics.csv:8 (row MEAN)	0.876117 / 0.758496 / +0.117621
Packet-level ROC-AUC (Tenko / Kitsune, 6 attacks)	ciciot2023_per_class_metrics.csv (AUC col, Kitsune & Tenko rows)	Table 2
Packet-level mean AUC 0.883 / 0.893	ciciot2023_per_class_metrics.csv (mean over 6)	0.883 / 0.893
Dictionary packet EER (Tenko < Kitsune)	ciciot2023_per_class_metrics.csv:22, 23 (EER col)	0.284700 vs. 0.318927
Tenko op-point F1 (strong-4 mean / all-6 mean)	cic_baselines_comparison.md (§2 derivations)	0.884 / 0.47
Sukanya & Raja 2025 (reported): overall ROC-AUC / F1 (VAE+IF)	cic_baselines_comparison.md:44, 99 (their Tables 4/8)	0.8947 / 0.55
Sukanya VAE / AE per-attack mean ROC-AUC	cic_baselines_comparison.md:45, 46 (their Table 6)	0.935 / 0.905
Ramkumar <i>et al.</i> 2025 (reported): AUC-PR / F1 mean (8 rows)	cic_baselines_comparison.md:47 (their Table 3)	0.967 / 0.795
Ullah <i>et al.</i> 2025 (reported, supervised): acc / F1 / AUC-ROC	baseline_comparison_ullah2025.md:54	0.992 / 0.992 / 0.992
Latency: Kitsune 7-28 pkts vs. Tenko thousands; pattern_window_size= 100	path_to_clear_win.md:76-82; _win_latency.py	honest loss
Robustness of source-level win (aggregation / attacker-set / tanh)	path_to_clear_win.md:127-132	+0.065-0.153